B-Tree

- Balanced m-way tree
- Generalization of BST in which a node can have more than one key and more than 2 children.
- All leaf nodes must be at same level.
- A B-Tree of order "m" has the following properties:

→ Every node has a maximum "m" children.

→ Min. children:

leaf → 0

root → 2

internal nodes → $\lceil \frac{m}{2} \rceil$.

→ Every node has maximum "m-1" keys.

→ Minimum keys:

Root node → 1

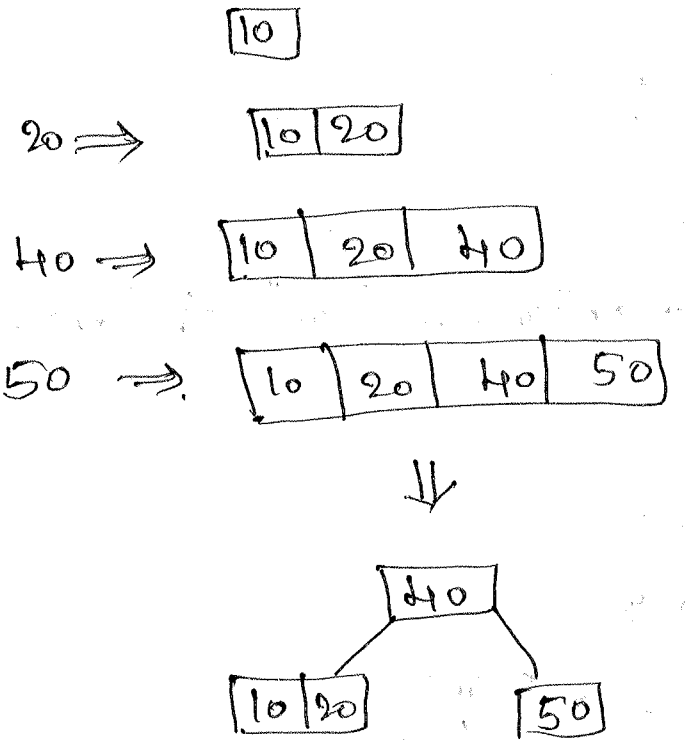
All other nodes → $\lceil \frac{m}{2} \rceil - 1$.

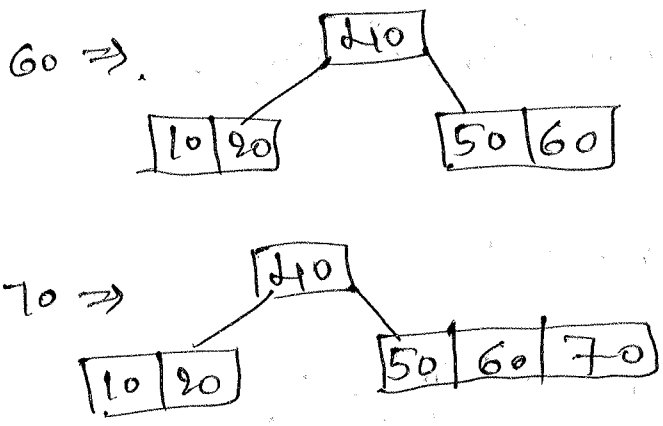
Example:

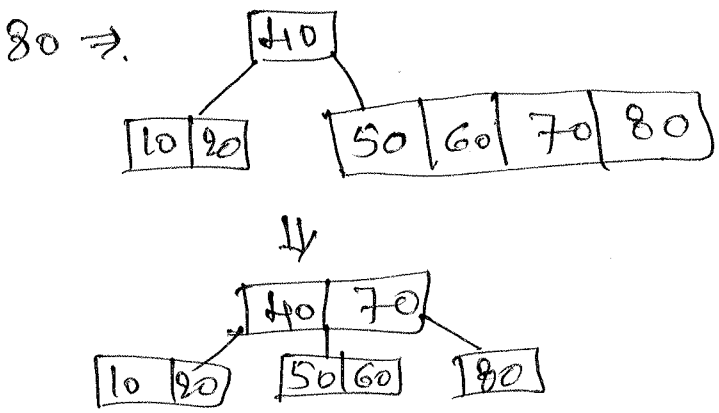
order $m = 4$

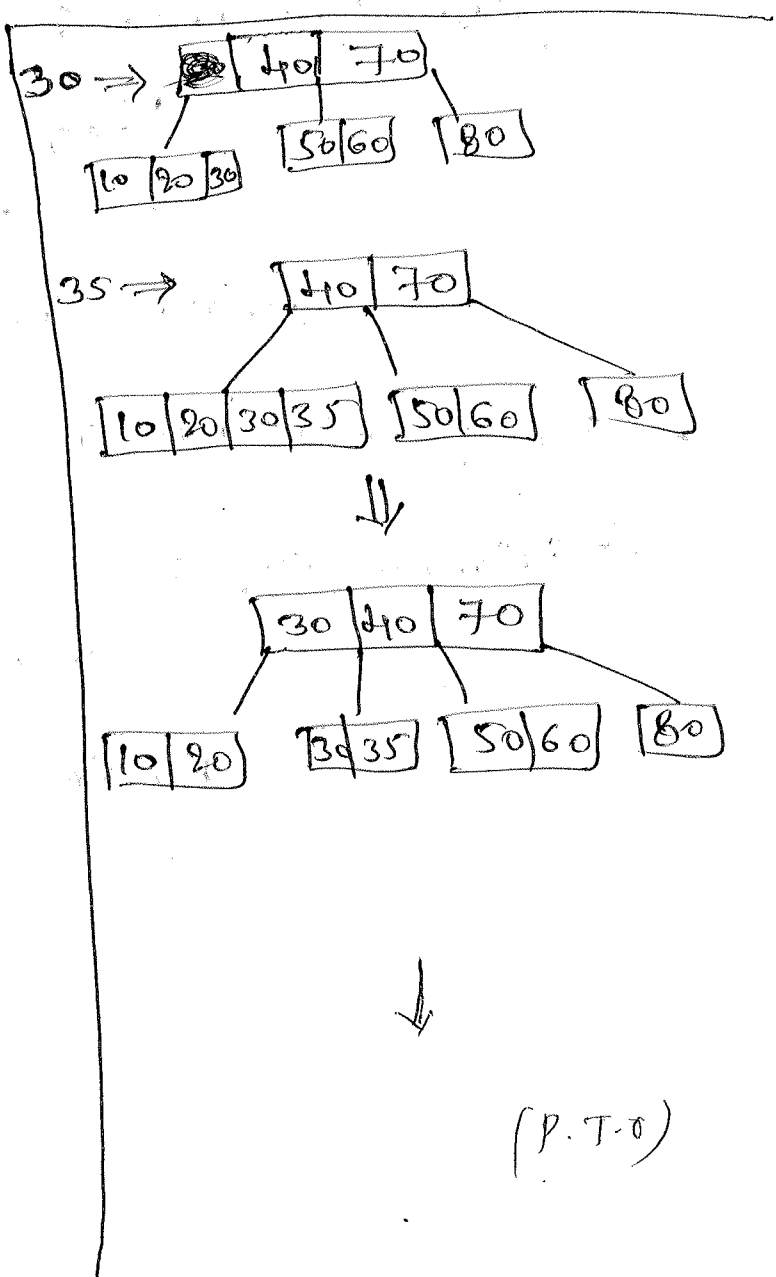
Keys : 10, 20, 40, 50

Max no. of keys in a node = 3



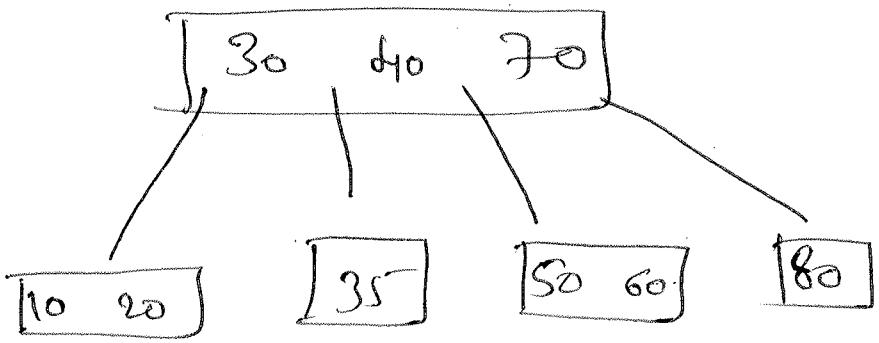




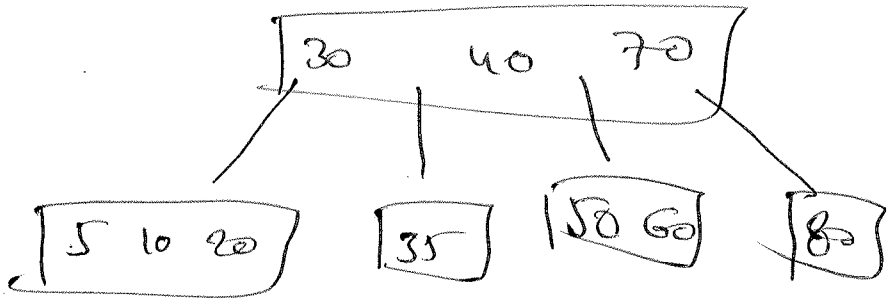


(P.T.O)

~~B Tree~~

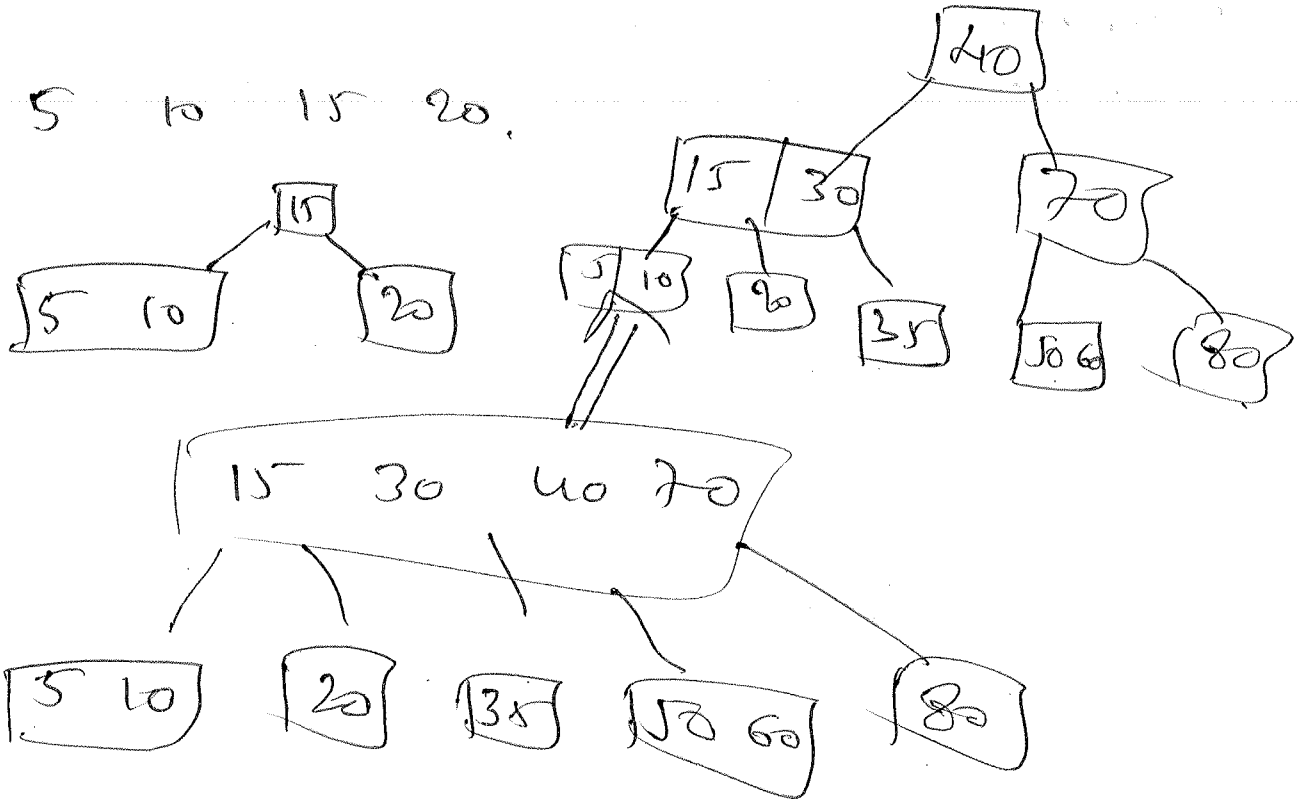


Insert 5



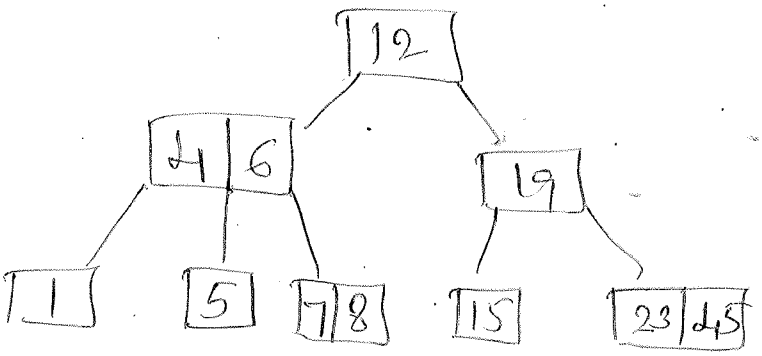
Insert 15

5 10 15 20.



B-Tree, $m = 3$

Values: 12, 23, 6, 8, 15, 19, 45, 1, 4, 7, 5

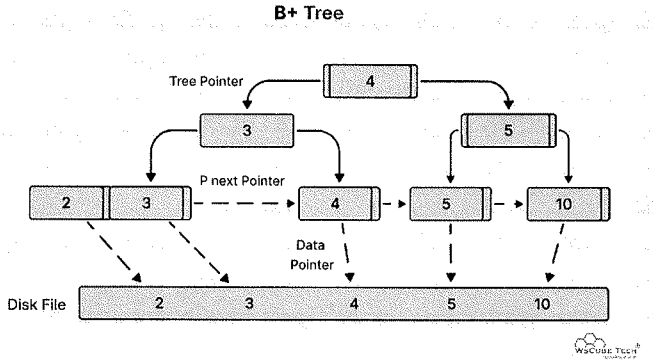


operations

- Insertion
- Deletion
- search

B+Tree

A B+ Tree is a type of data structure used in computer science for efficient data storage and retrieval. Unlike B Tree, the B+ Tree stores all values in the leaf nodes and use internal nodes only for indexing.



Properties of B+ Tree

•**Balanced Tree Structure:** B+ Trees are balanced, meaning all leaf nodes are at the same level.

•**Leaf Nodes Store All Values:** All actual data values are stored in the leaf nodes.

•**Internal Nodes for Indexing:** Internal nodes contain only keys and pointers to child nodes.

•**Linked List of Leaf Nodes:** Leaf nodes are linked together in a linked list.

•**Order of the Tree (m):** The order (m) of a B+ Tree defines the maximum number of children each internal node can have.

•For a B+ Tree of order m:

- Each internal node can have at most m children.
- Each internal node (except the root) must have at least (m/2) children.
- Each internal node must have between (m/2) - 1 and m - 1 keys.
- Each leaf node must have between (m/2) and m values.

Operations on B+ Trees

1. Insertion Algorithm:

1.Start from the root and find the appropriate leaf node where the key should be inserted.

2.Insert the key in the leaf node in sorted order.

3.Split the Node (if necessary):

- If the leaf node has more than $m-1$ keys after insertion, split the node into two.
- Promote the middle key to the parent node.
- If the parent node also overflows, repeat the splitting process up to the root.

4.Ensure that leaf nodes are linked correctly after any split.

2. Deletion Algorithm:

1.Start from the root and find the leaf node containing the key.

2.Remove the key from the leaf node.

3.Merge or Redistribute Nodes (if necessary):

- If the leaf node has fewer than $\lceil m/2 \rceil$ keys after deletion, merge or redistribute nodes.
- If merging, remove the corresponding key from the parent and merge nodes.
- If the parent node underflows, repeat the process up to the root.

3.Searching Algorithm:

1.Begin the search from the root node.

2.Compare the key with the keys in the current node:

- If the key is less than a key in the node, follow the corresponding child pointer.
- If the key is greater, move to the next key or child pointer.

3.Continue this process until you reach the leaf node.

4.Look for the key in the leaf node.

B+Tree

* Data is stored at leaf nodes.
* each leaf node is connected to its next leaf node.

25, 9, 4, 16, 1, 20, 13, 15, 10, 11, 12, 18.

order = 4

$\lceil \frac{4}{2} \rceil \leq$ No. of children ≤ 4

$1 = \lceil \frac{4}{2} \rceil - 1 \leq$ No. of keys $\leq 4-1 = 3$

Node gets merged if count goes below

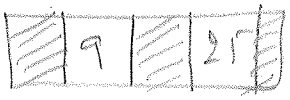
Node gets split when 4th elt comes.

No. of children = No. of keys + 1

step ①: Insert 25



step ②: Insert 9

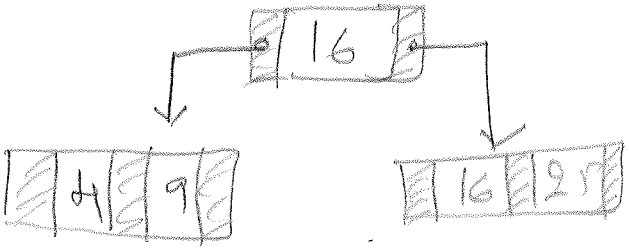


step ③: Insert 4

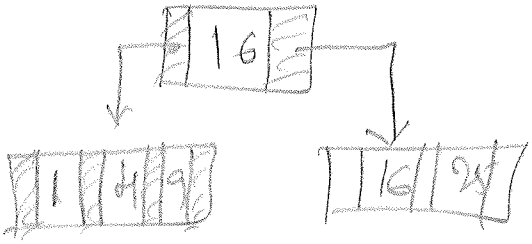


step ④: Insert 16

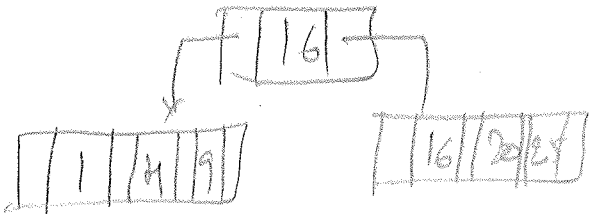
elements 4, 9, 16, 25



step ⑤: Insert 1:



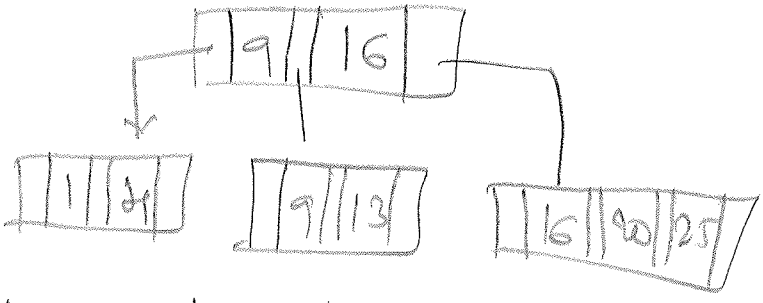
step ⑥: Insert 20.



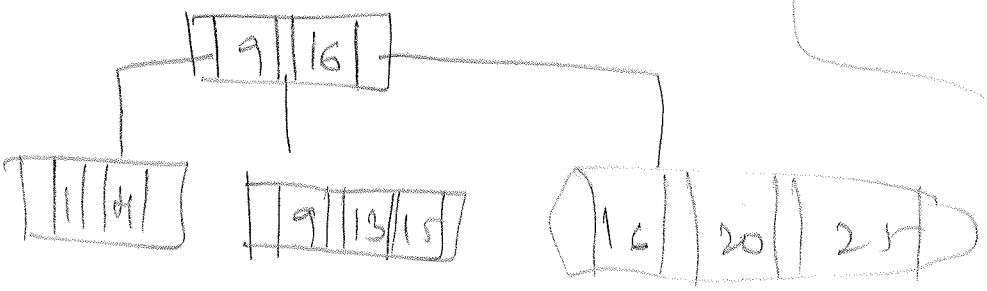
step ⑦ Insert 13

1, 4, 9, 13.

Split the node, New parent: 9

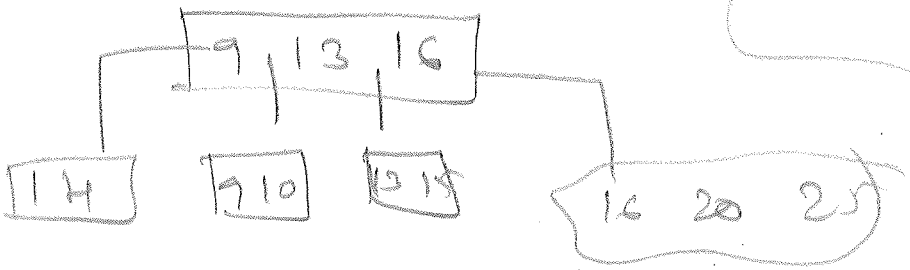


step ⑧ : Insert 15



step ⑨ : Insert 10

9, 10, 13, 15



step 10 : 11



step ⑪ : 12

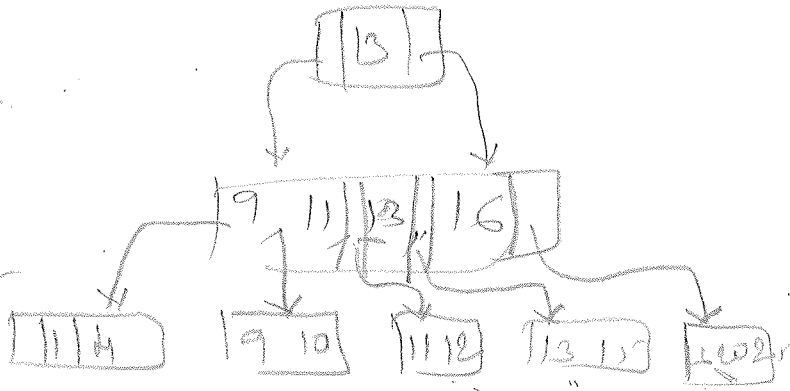
9, 10, 11, 12

New parent

Next node

9, 11, 13, 16

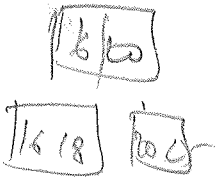
New parent



step ⑫ Insert 18.

overflow in 5th child node.

13

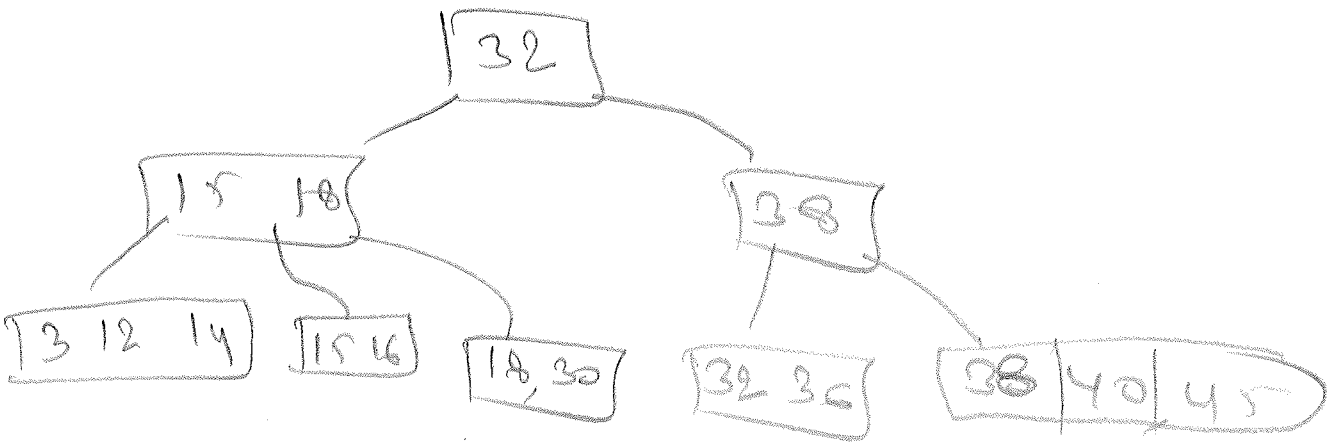


B^+ Tree

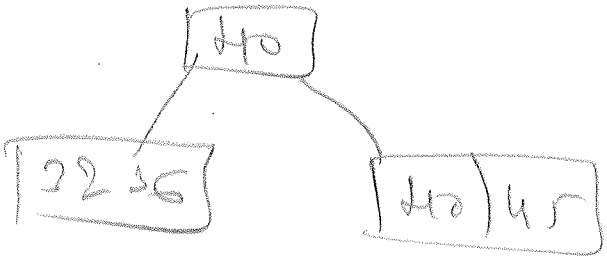
(2) [3]

3, 18, 14, 30, 32, 36, 15, 16, 12, 40, 38, 45

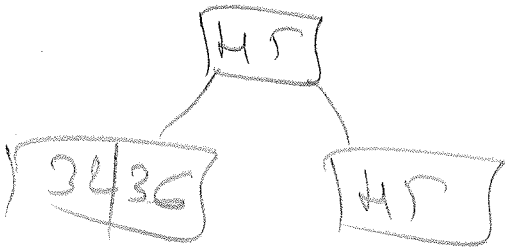
$m = 4$



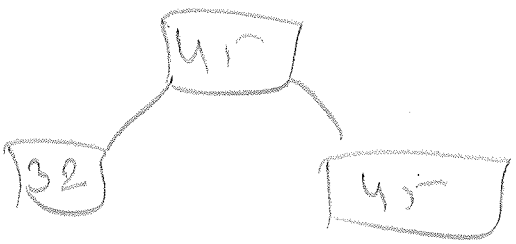
Delete : 38, 40, 36, 15, 18.



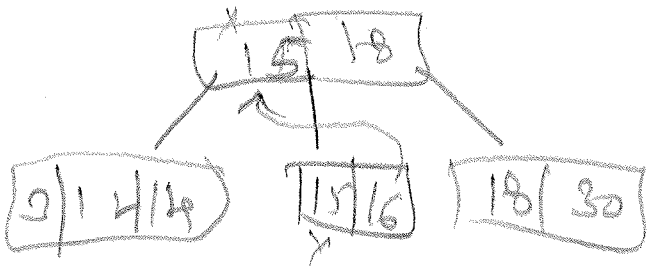
40

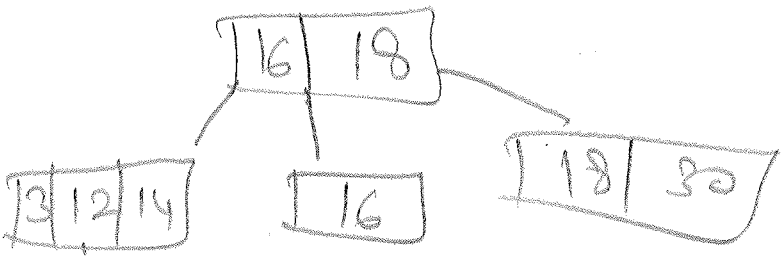


36 :-

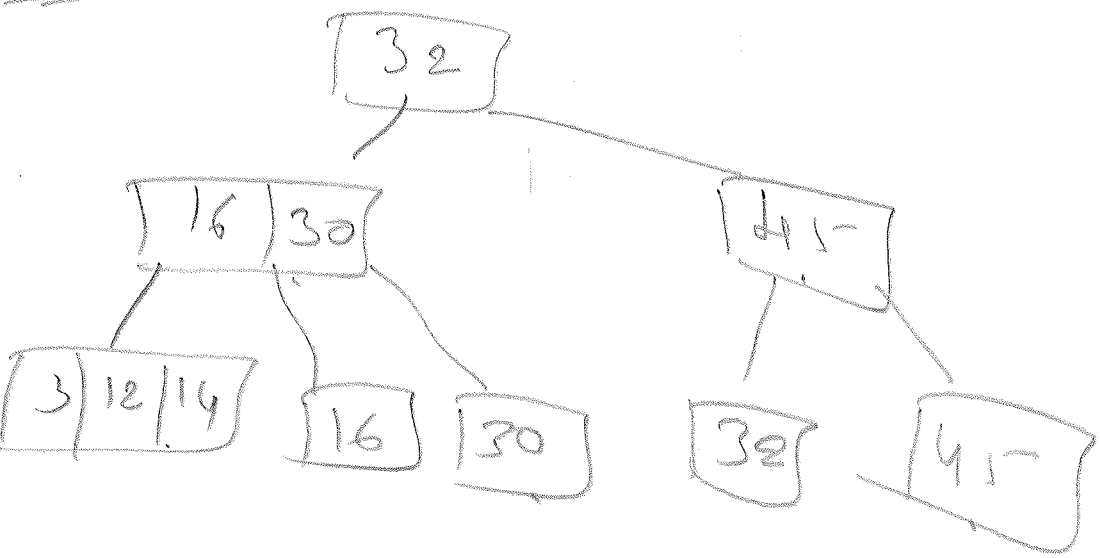


١٢ :-

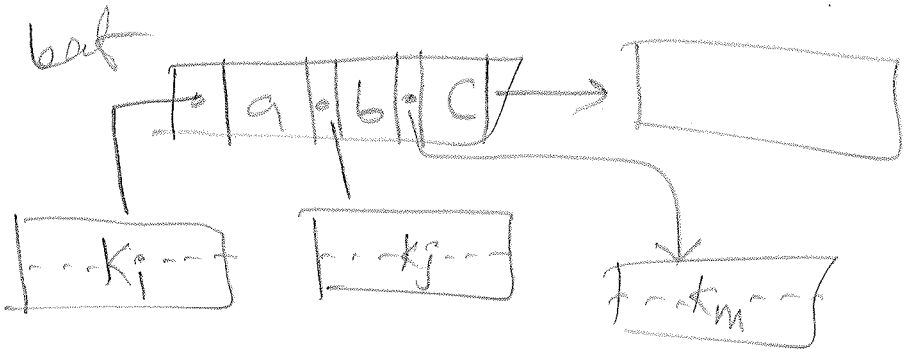




delete 18 :-



Note :

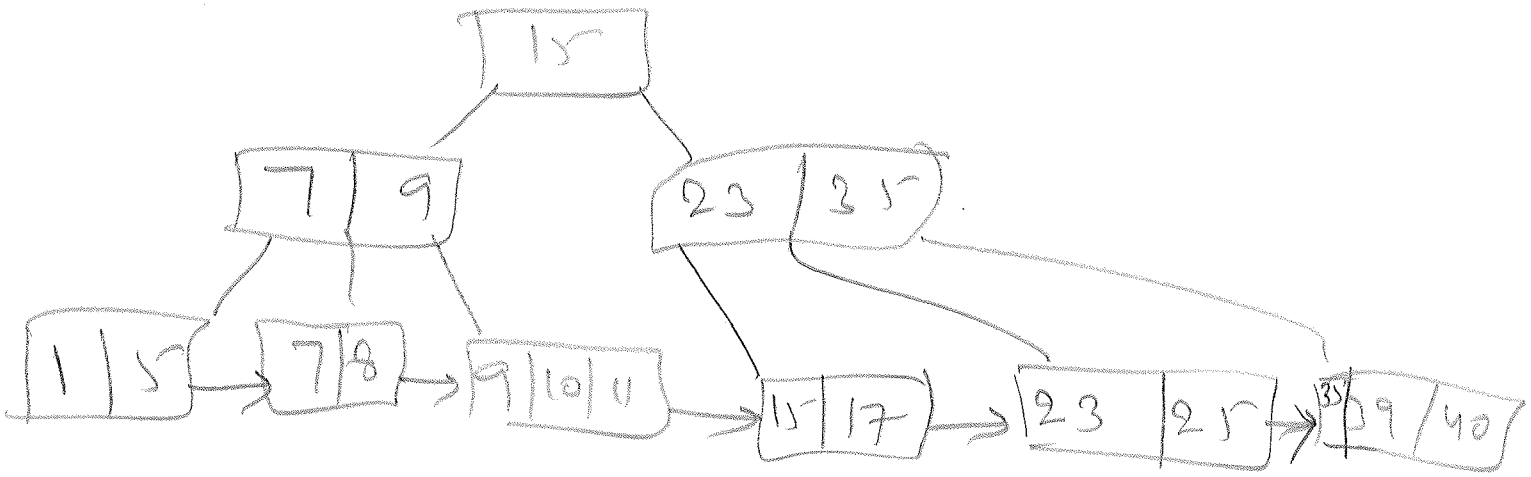


$k_i < a$ $a \le k_j \le b$ $b \le k_m < c$ . . .

B+ Tree of order 5

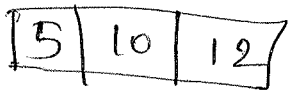
Ex: 7, 10, 1, 23, 5, 15, 17, 9, 11, 39, 35, 8, 40, 25.

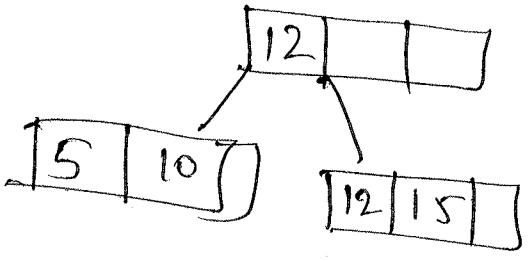
m = 5.



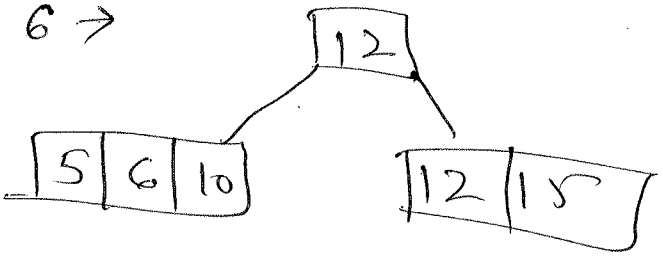
B+Tree

5, 10, 12, 15, 6, 21, 45, 36, 65, 14 order → 4

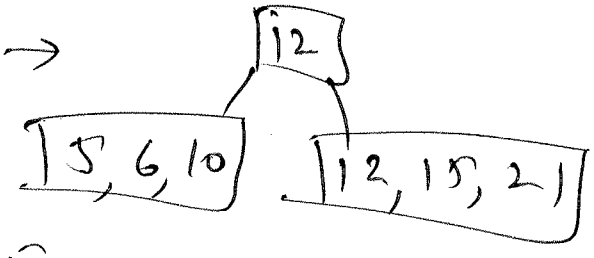




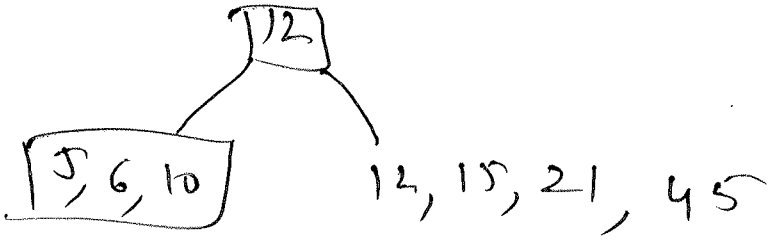
6 →

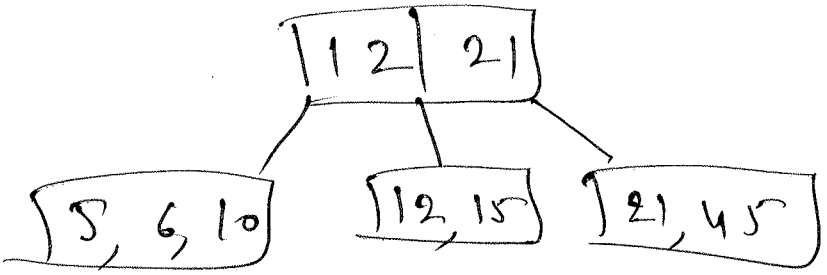


21 →

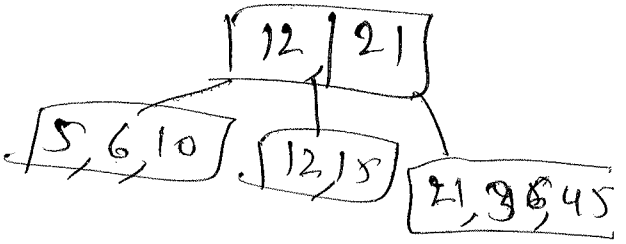


45 →

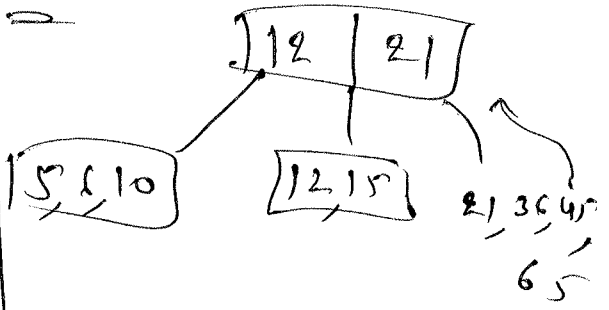


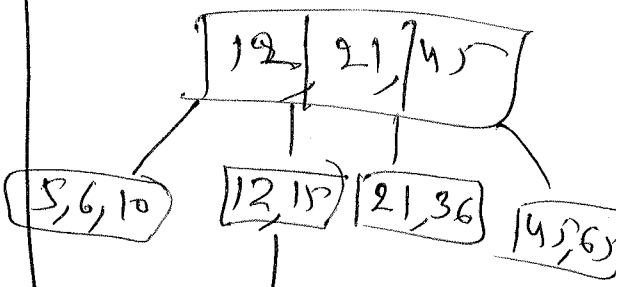


36 →

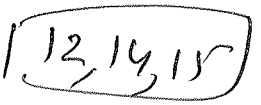


65 →





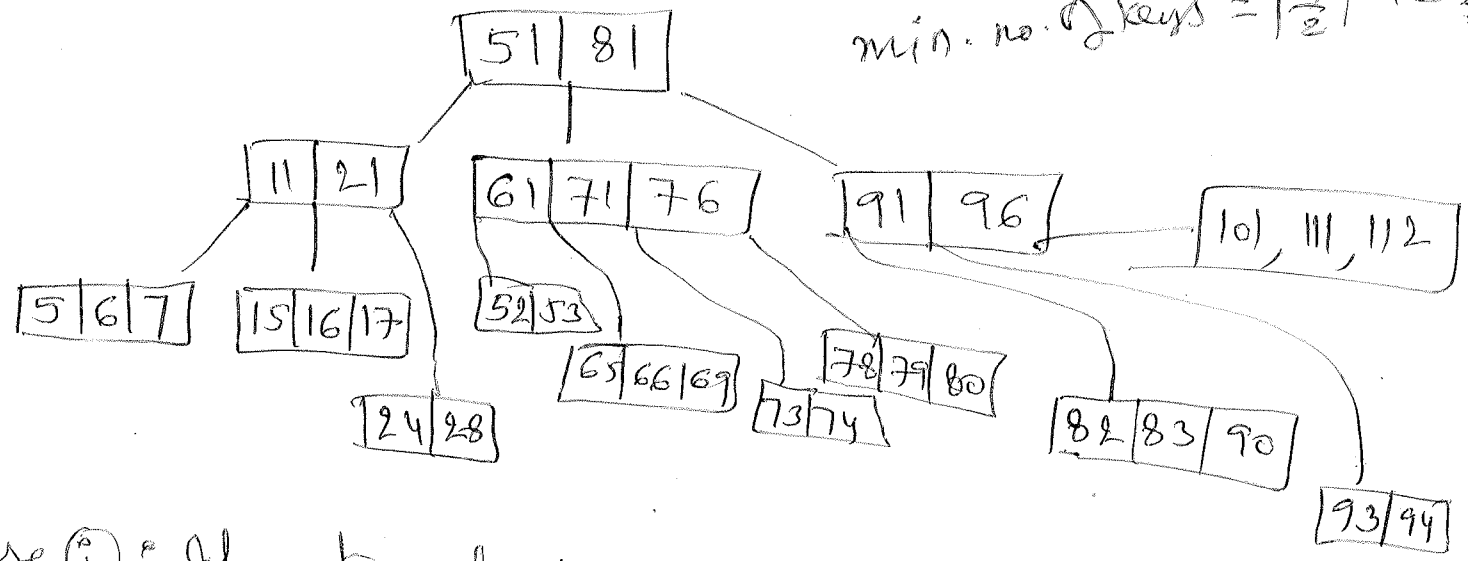
14 →



Deletion from a B-Tree

$m = 5$

min. no. of keys $= \lceil \frac{m}{2} \rceil - 1 = 2$



Case (i) : If a target key is a leaf node.

$m = 5$

min no. of keys in a Root = 1

" " " in any other node $= \lceil \frac{m}{2} \rceil - 1 = 2$

Max " " a node $= m - 1 = 4$.

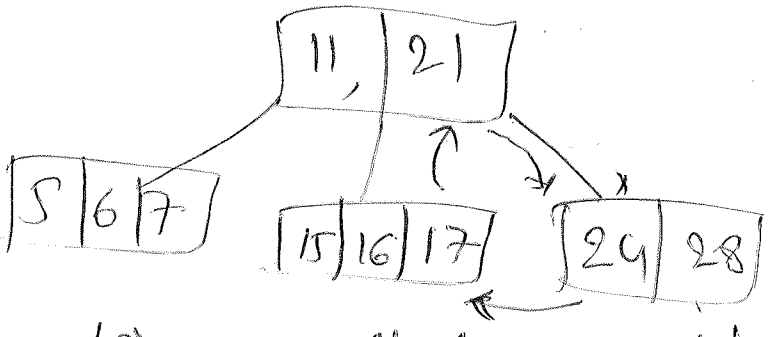
Delete (65)

Simply delete. (∵ no violation of properties)

Delete (24)

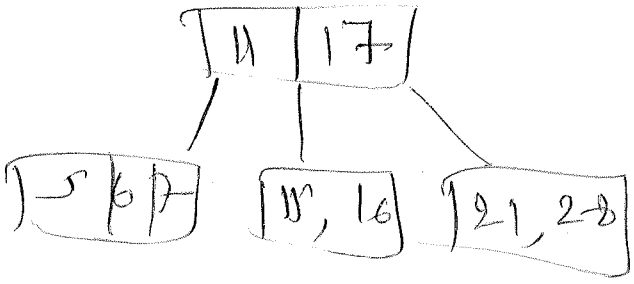
(borrow from left sibling)

min. no. of keys in a node is 2



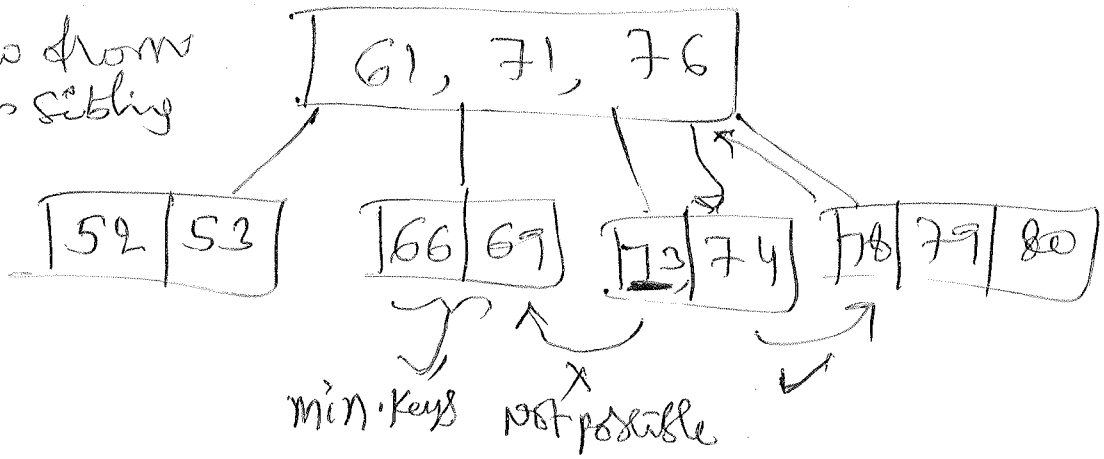
voilates B-Tree property that min keys should be 2

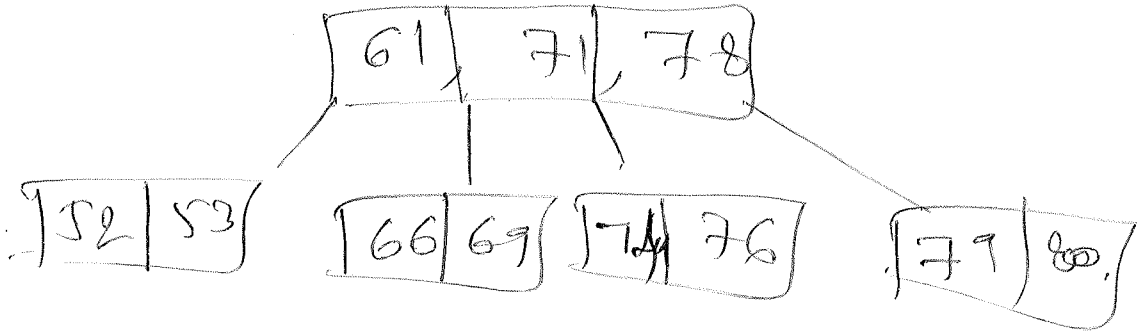
borrow an elt from left/Right sibling.



delete (73):

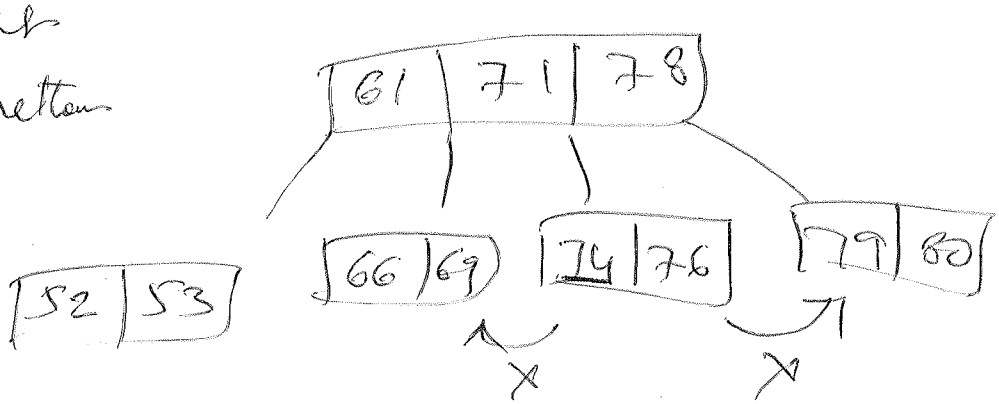
borrow from right sibling



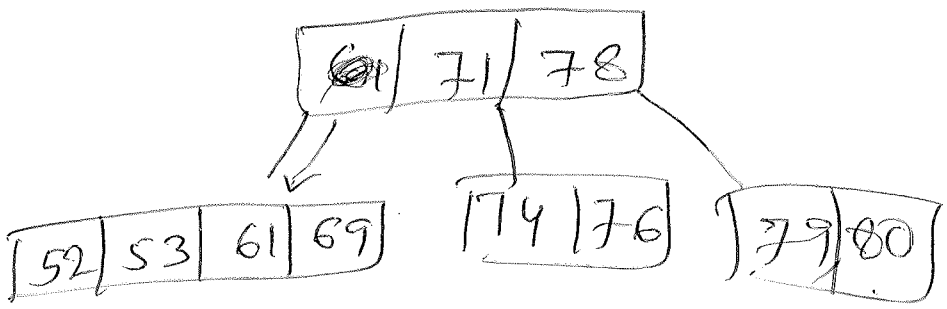


Delete (66):

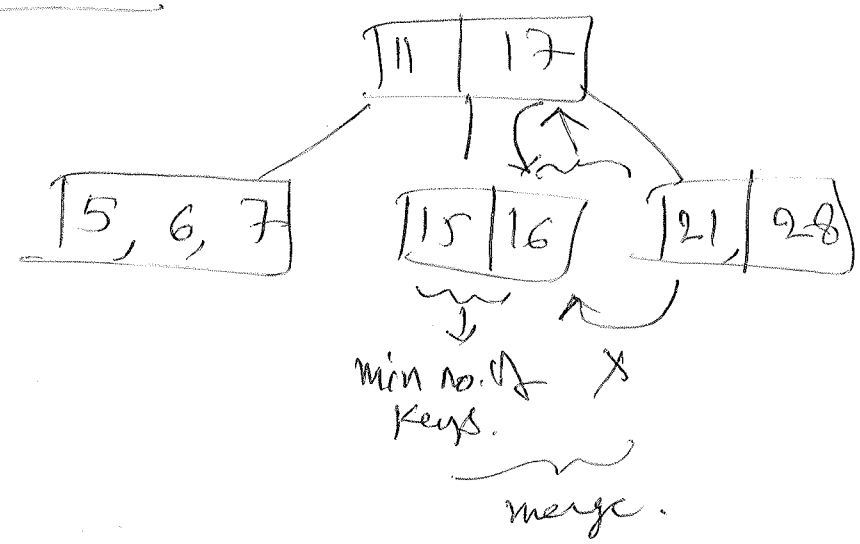
neighbor left/right sibling has more than 2 keys

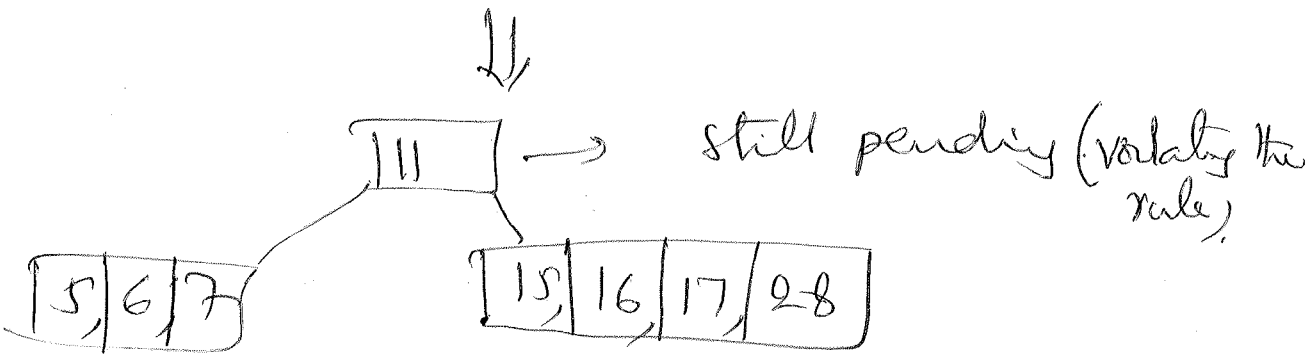


Now merging takes place.

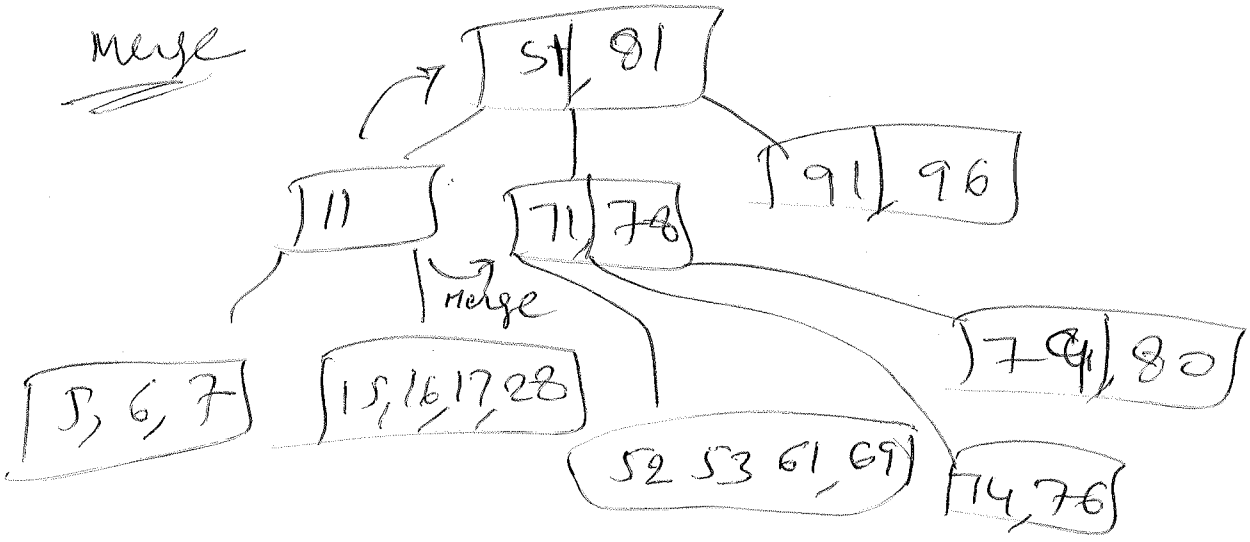


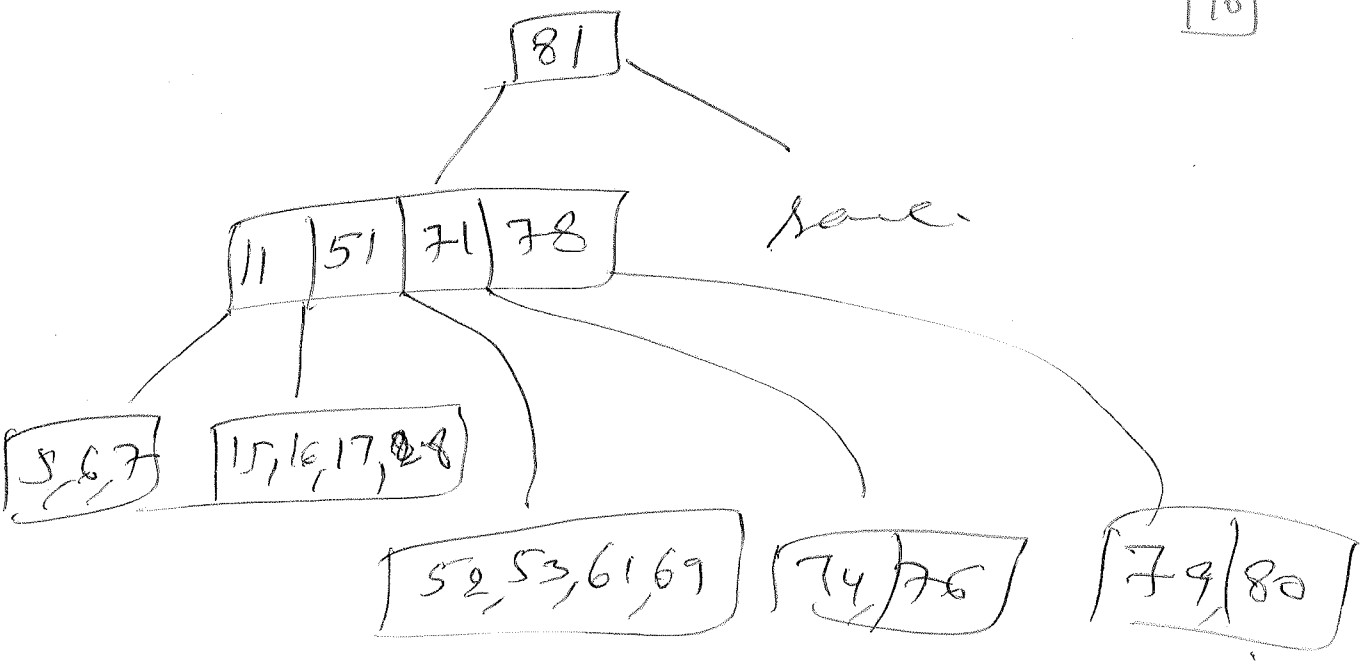
Delete (21) :



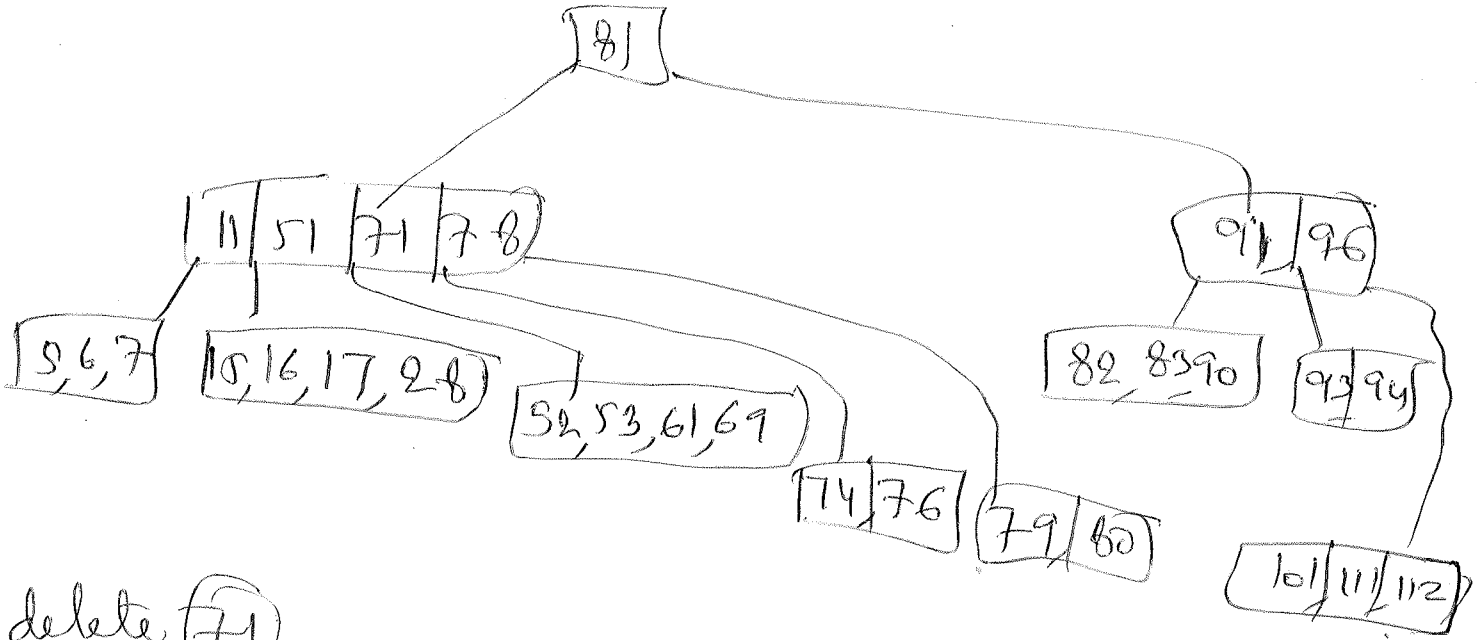


Merge

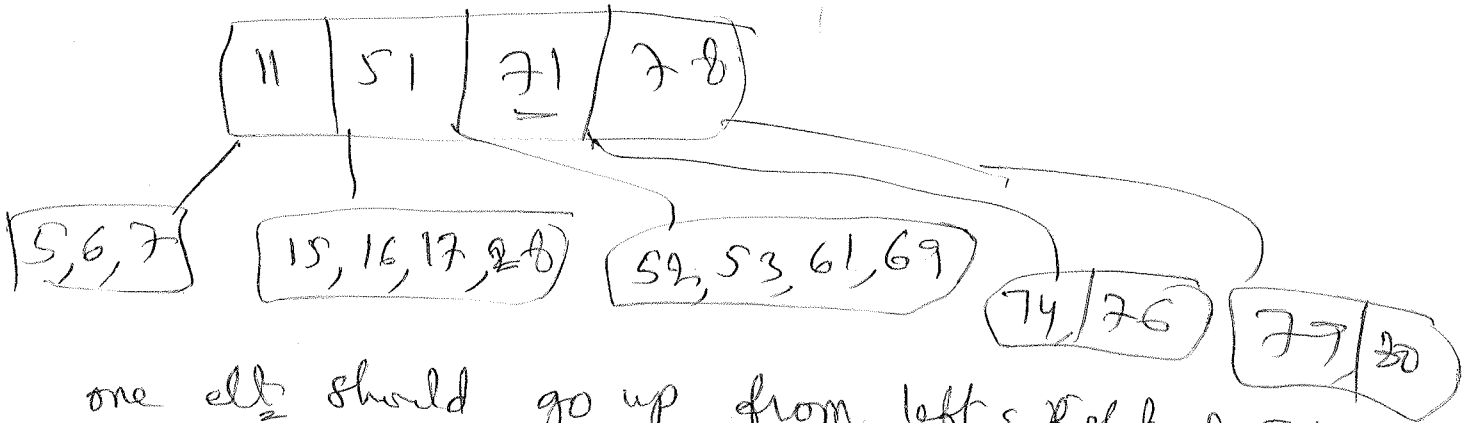




(6) If a target key is internal node.

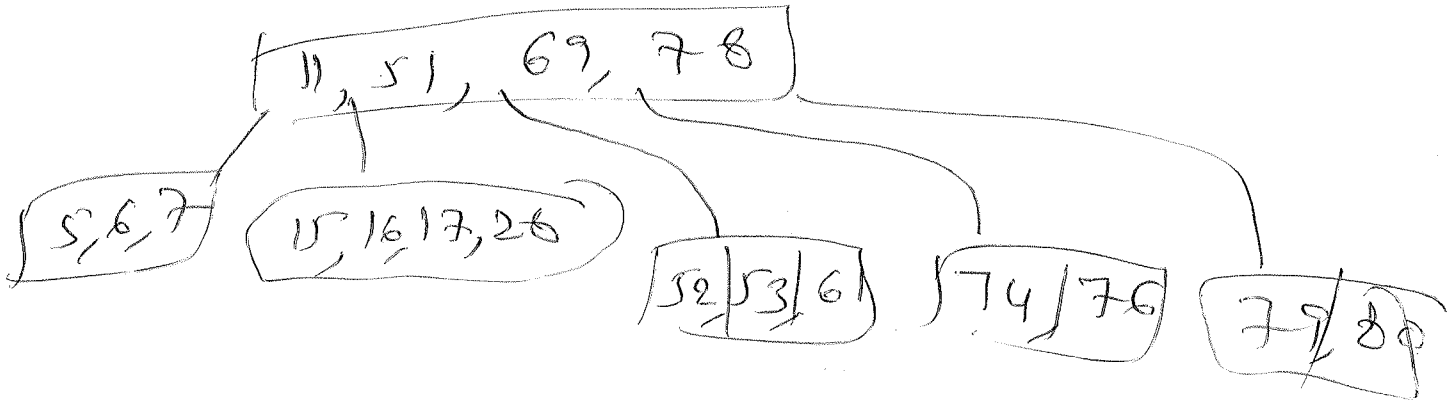


delete (71)

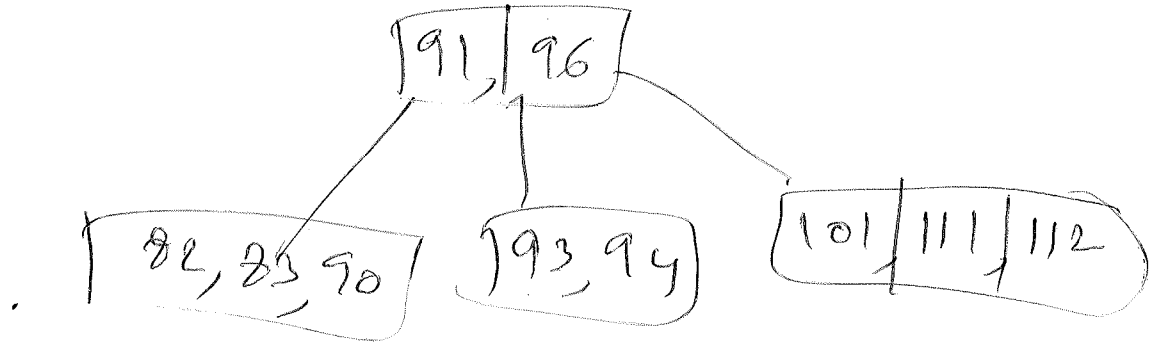


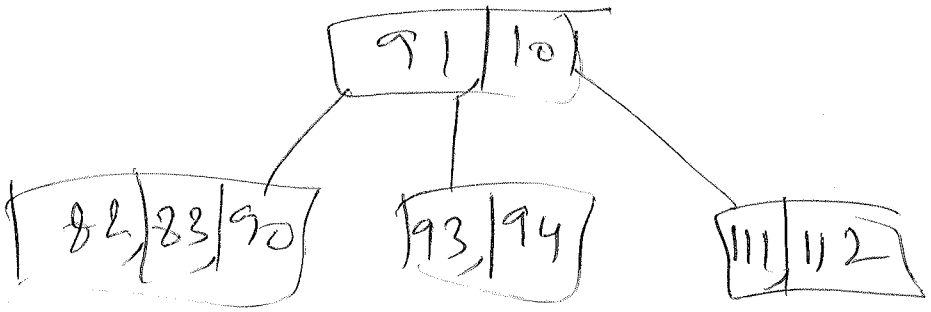
one elt should go up from left & Right of 71.

We can not borrow from Right child as it has min no. of elts.

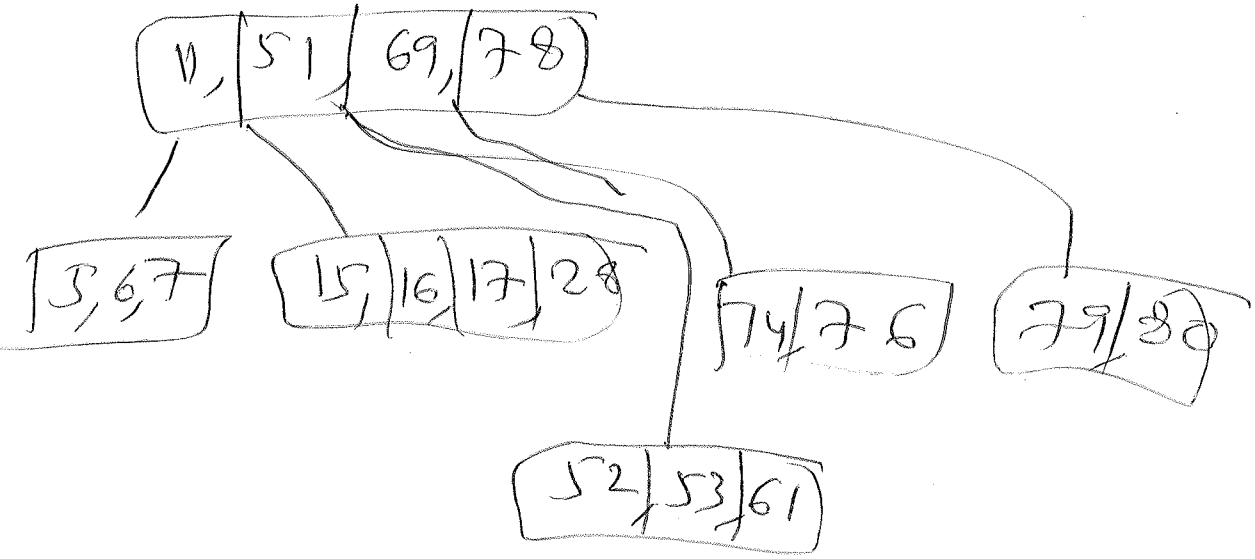


delete 96





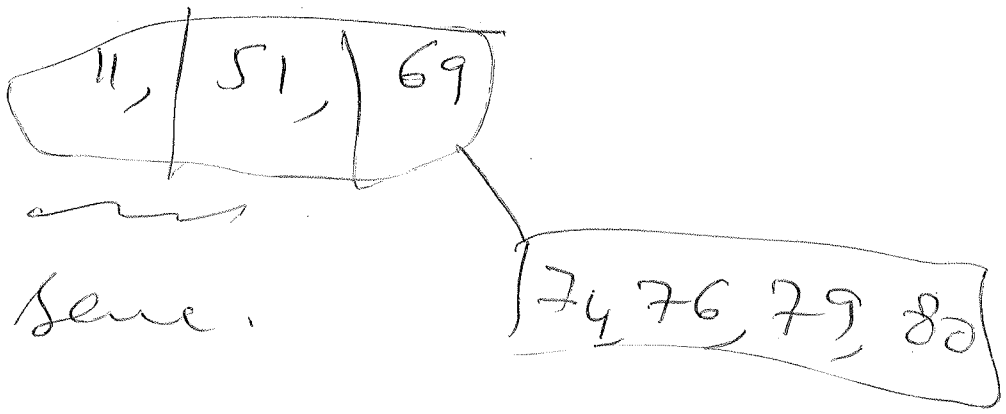
delete (78)



to delete 78 we need to borrow an elt from either left / Right subtree (child)

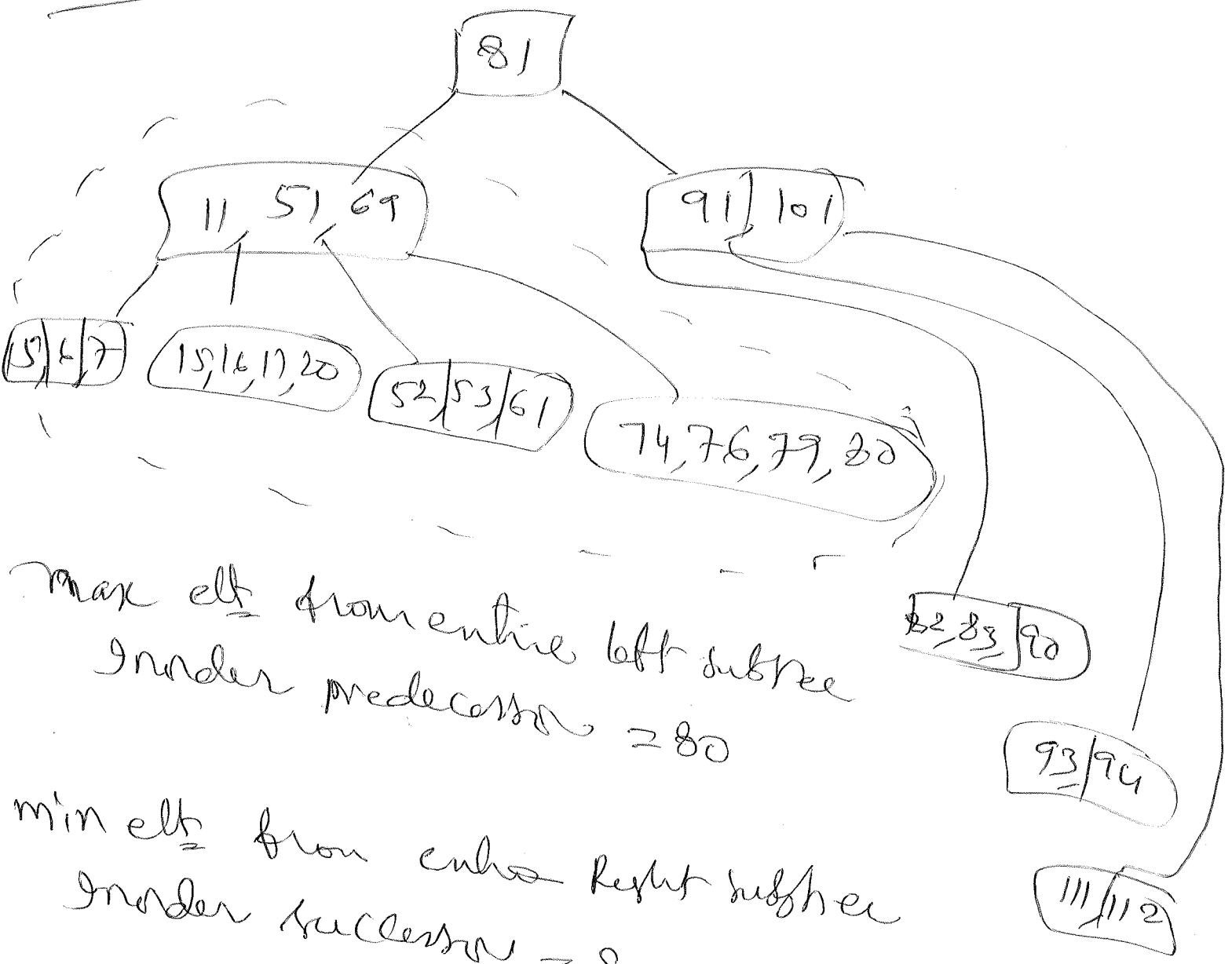
but none of them can give as they have min. elts.

In this case we need to merge both children.



Same.

delete (81)

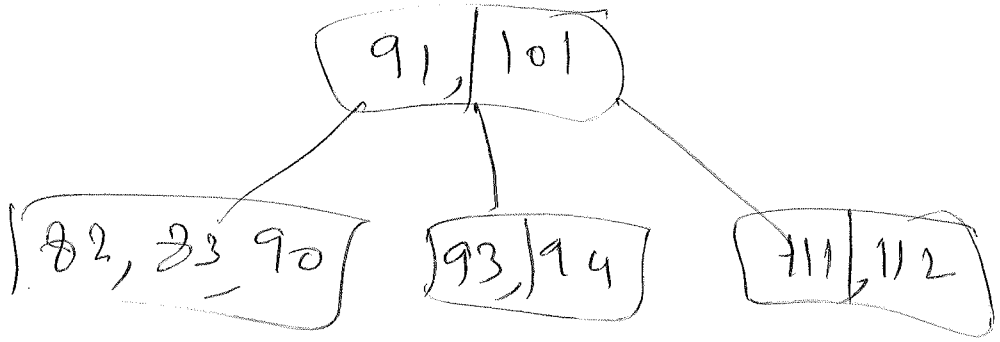


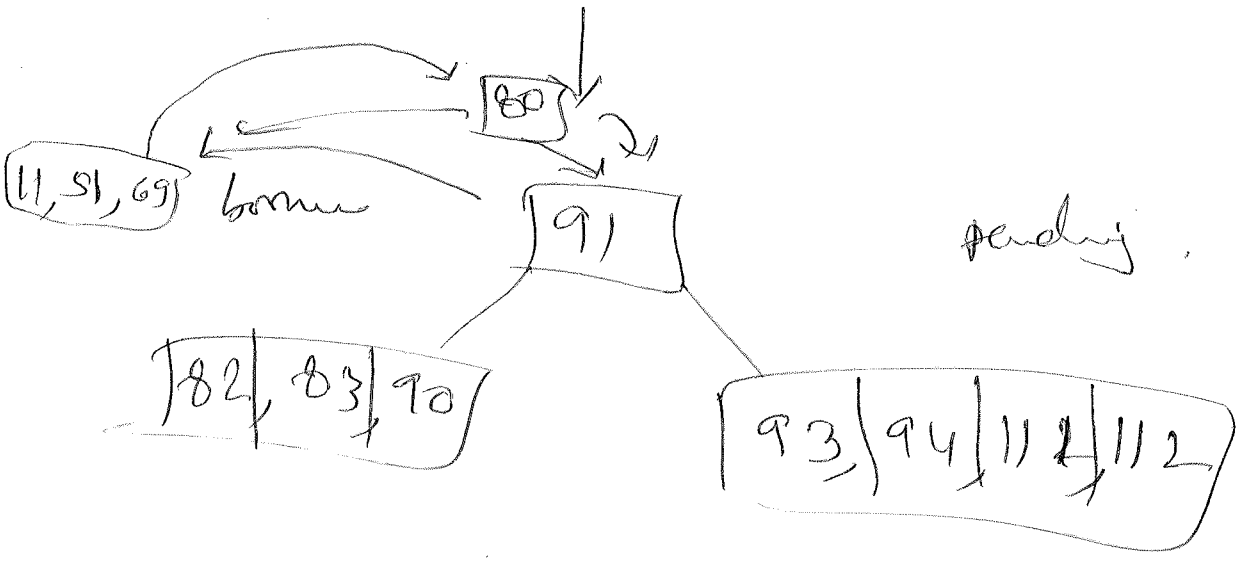
max elt from entire left subtree
Inorder predecessor = 80

min elt from entire Right subtree
Inorder successor = 82

80 → root node.

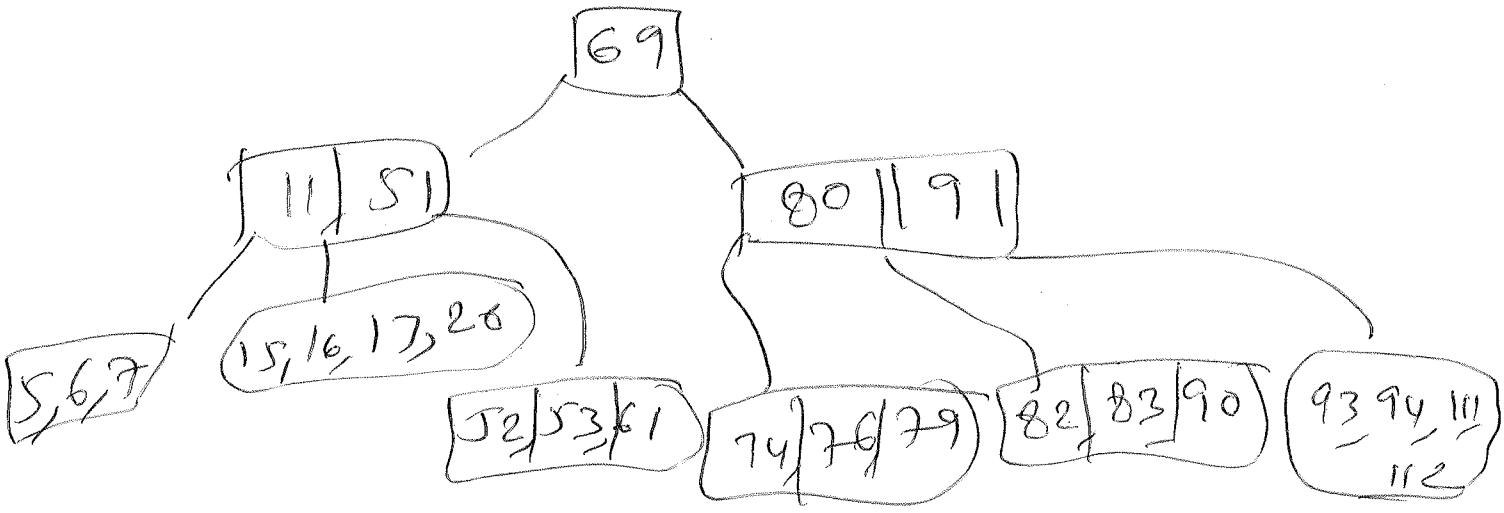
delete (101)



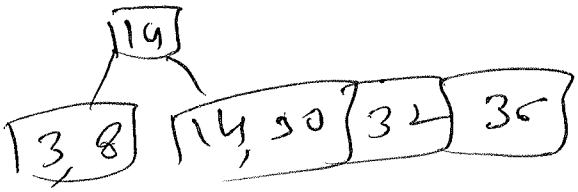


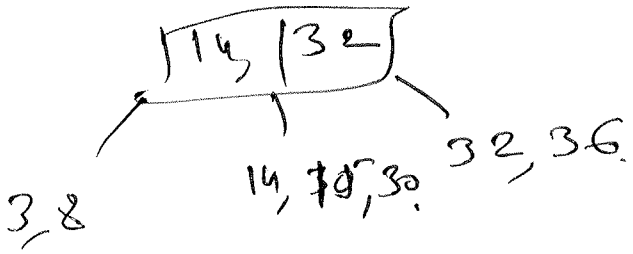
(91) → is not root node. It should contain "2" elts.

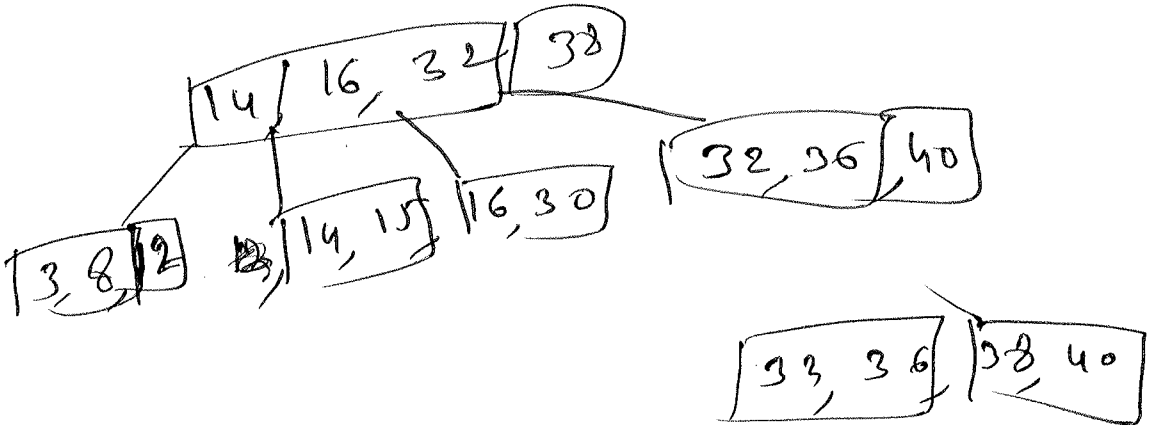
borrow from left sibling.

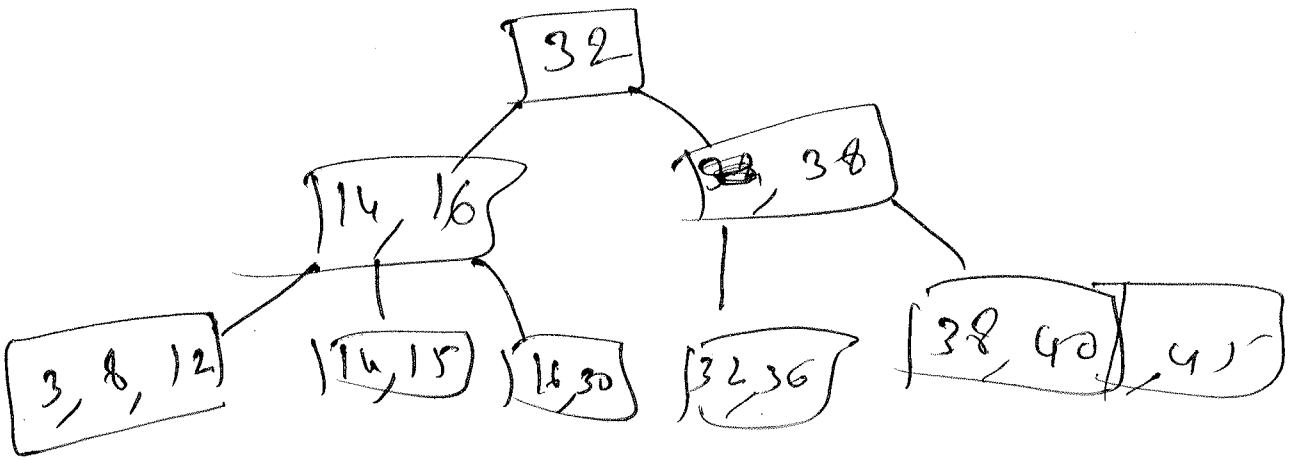


3, 8, 14, 30


14
3, 8
14, 30
32
36


14, 32
3, 8
14, 30
32, 36


14, 16, 32
38
32, 36, 40
3, 8, 12
14, 15
16, 30
32, 36
38, 40


32
14, 16
38
3, 8, 12
14, 15
16, 30
32, 36
38, 40